

TP 2 – Stratégies d'authentification

Objectifs de la séance

- Prendre en main et maîtriser les concepts de base de Keycloak
- Maîtriser l'intégration de Keycloak à une application existante

Dans ce TP, une application **Flask** simple utilisant Docker est fournie sur le repository GitHub suivant :

<https://github.com/pierreavn/swc-tp-keycloak>

Le **compte-rendu du TP** sera à uploader (PDF) sur e-campus avant le **30 septembre 23h59**.

Il devra détailler de manière **concise et claire** l'ensemble des **points marqués avec l'icône** : 📌

→ **A chaque fois qu'une validation est attendue, sollicitez l'encadrant de TP** 😊

Démarrage de l'application

1. Cloner le repository du projet ci-dessus et démarrer le projet avec la commande :

```
Shell
docker compose up
```

2. Accéder à l'application sur le port 8081 et vérifier son bon fonctionnement :

<http://localhost:8081>

💡 Si le serveur Keycloak plante et s'arrête quelques secondes après le démarrage, il est très probable que Docker n'ait pas les permissions pour écrire les données de Keycloak sur votre disque. Pour cela, modifiez les permissions avec la commande :

```
sudo chown -R 1000:1000 ./keycloak-data
```

Configuration de Keycloak

3. Accéder à l'interface Keycloak sur le port 8080 et vérifier son bon fonctionnement :

<http://localhost:8080>

4. Se connecter avec les identifiants par défaut configurés dans le **docker-compose.yml** :

Username : **admin** Password : **admin**

💡 Un seul serveur Keycloak permet de gérer l'authentification de plusieurs applications différentes. Un **realm** dans Keycloak est un découpage logique pour gérer une application. Chaque realm aura sa propre liste d'utilisateurs, ses propres rôles et permissions, son propre thème, ...

Par défaut, Keycloak crée le realm **master**, qui correspond à la gestion de l'authentification sur Keycloak lui-même. Ainsi, la liste des utilisateurs dans le realm **master** correspond aux utilisateurs pouvant s'authentifier à l'administration de Keycloak.

5. Créer un nouveau realm "**webapp**", qui correspond à notre application



Un **client** dans Keycloak correspond à une **brique de notre application qui a besoin d'interagir avec Keycloak** (que ce soit pour authentifier un utilisateur, vérifier un JWT, ...). Actuellement, notre webapp est composée d'un frontend (HTML + JavaScript) et d'un backend (Python Flask). Nous allons donc créer deux clients nommés **webapp-frontend** et **webapp-backend**.

6. Dans ce nouveau realm, **créer un nouveau client "webapp-frontend"**, avec :
 - a. **Client authentication** → **OFF** (car il s'agit d'un client public)
 - b. Authentication flow → **Standard flow** (cf. cours)
 - c. Root URL → URL de l'application : **http://localhost:8081**
 - d. Home URL → URL de l'application : **http://localhost:8081**
 - e. Valid redirect URIs → **http://localhost:8081/***
 - f. Web origins → **http://localhost:8081**
7. Toujours dans ce même realm, **créer un nouveau client "webapp-backend"**, avec :
 - a. **Client authentication** → **ON** (car il s'agit d'un client privé)



Cette option va générer un mot de passe secret que le client devra fournir à Keycloak pour interagir avec lui.

- b. Authentication flow → **Direct access grants** (cf. cours)
 - c. Root URL → URL de l'application : **http://localhost:8081**
 - d. Home URL → URL de l'application : **http://localhost:8081**
8. Une fois le client créé, aller dans l'onglet **Credentials**, et copier le **Client Secret** dans la variable d'environnement **KEYCLOAK_BACKEND_CLIENT_SECRET** du fichier **docker-compose.yml**
 9. Afin de prendre en compte les modifications de variable d'environnement, **arrêter le docker compose** et démarrer à nouveau :

Shell

```
# appuyer sur CTRL+C, puis à nouveau :  
docker compose up
```

10. Toujours dans le realm webapp, **créer un nouveau Realm role "is-admin"**
11. Toujours dans le realm webapp, **créer un nouvel utilisateur** en remplissant le nom d'utilisateur, l'e-mail, le prénom et le nom de famille.
12. Une fois l'utilisateur créé, aller dans l'onglet **Credentials** et **définir un mot de passe**.



Si vous activez l'option **Temporary**, il sera demandé à l'utilisateur de définir un nouveau mot de passe lors de sa première connexion.

13. Toujours sur ce même utilisateur, aller dans l'onglet **Role mapping** et assigner le **Realm role "is-admin"** précédemment créé.



La configuration de Keycloak est terminée !

Intégration de Keycloak dans le frontend de l'application



Pour simplifier son intégration, Keycloak met à disposition un **code JavaScript officiel** permettant de **gérer toutes les requêtes nécessaires au Standard Flow**.

14. Modifier le code HTML de la page de connexion (**webapp/templates/index.html**) pour **importer le code JavaScript de keycloak-js**, la librairie officielle de Keycloak :

HTML

```
<script  
src="https://cdn.jsdelivr.net/npm/keycloak-js@latest/dist/keycloak.min.js"></script>
```

15. Développer et ajouter à la suite votre **code JavaScript pour utiliser la librairie Keycloak** :

JavaScript

```
<script>  
  const keycloak = new Keycloak({  
    url: "http://localhost:8080", // URL du serveur Keycloak  
    realm: "webapp",           // Nom du realm précédemment créé  
    clientId: "webapp-frontend", // Nom du client frontend  
  });  
  
  keycloak.init({ onLoad: "login-required" })  
    .then((authenticated) => { // Action lorsque le standard flow est terminé :  
      if (authenticated) {  
        // 👉 [TODO] si l'utilisateur est connecté :  
        //  
        // 1. Afficher à la place du texte du div[id="user"] :  
        //    `Bienvenue ${keycloak.tokenParsed.email}`  
        //  
        // 2. Appeler keycloak.logout() au clic sur le button[id="logoutBtn"]  
      }  
    })  
    .catch((error) => {  
      console.error("Failed to initialize Keycloak :", error);  
    });  
</script>
```

16. Tester l'authentification de bout en bout en accédant à l'application sur <http://localhost:8080>
→ **Faire valider le fonctionnement de l'authentification**

17. Modifier le code JavaScript pour **effectuer un appel API une fois le JWT obtenu** :

JavaScript

```
<script>
...

keycloak.init({ onLoad: "login-required" })
  .then((authenticated) => {
    if (authenticated) {
      // 👉 [TODO] si l'utilisateur est connecté :
      //
      // 1. Utiliser fetch(...) pour faire un appel GET sur /api/account
      //    en fournissant l'access token (variable keycloak.token) dans le
      //    header "Authorization" de la requête et obtenir la balance (en €)
      //    de l'utilisateur connecté
      //
      // 2. Afficher à la place du texte du div[id="user"] :
      //    `Bienvenue ${keycloak.tokenParsed.email}, balance : {...} €`
      //
      // 3. Appeler keycloak.logout() au clic sur le button[id="logoutBtn"]
    }
  })
  .catch((error) => {
    console.error("Failed to initialize Keycloak :", error);
  });
</script>
```

18. Modifier la fonction **api_account** dans le **code Python de l'API** (webapp/app.py) pour :

- a. Récupérer le **JWT dans le header "Authorization"** de la requête
- b. **Vérifier la validité du JWT** et **extraire les informations de l'utilisateur authentifié** :

Python

```
keycloak_openid.userinfo("...jwt...")
```

c. Retourner la **balance associée à l'utilisateur** (utiliser la variable `users_balances`)

19. Tester l'authentification de bout en bout et l'affichage de la balance de l'utilisateur

➔ **Faire valider le fonctionnement de l'authentification**

🎉 **L'intégration de Keycloak à l'application est terminée !**



Si votre application utilise un **framework** pour le frontend (React, Angular, Vue, ...), vous pouvez utiliser directement le **package NPM de keycloak-js** :

<https://www.npmjs.com/package/keycloak-js>

Analyse du flux d'authentification

20. 🛠️ Établir un **diagramme de séquence du processus d'authentification Standard flow** sur l'application de bout en bout, en décrivant principalement :
 - a. la requête vers `http://localhost:8080/realms/webapp/protocol/openid-connect/auth` en décrivant ses paramètres (`client_id`, `redirect_uri`, `response_type`, ...)
et son code HTTP de réponse (en fonction de si l'utilisateur est connecté ou non)
 - b. la requête vers `http://localhost:8080/realms/webapp/protocol/openid-connect/token` en décrivant ses paramètres (`client_id`, `grant_type`, `code`, ...)
et le contenu de sa réponse (`access_token`, `expires_in`, ...)
21. Dans Keycloak, modifier la configuration du client **webapp-frontend** pour décocher le flux d'authentification **Standard flow** et activer à la place le flux d'authentification **Implicit flow**
22. Modifier également le **code JavaScript** pour utiliser le flux d'authentification **Implicit flow** :

JavaScript

```
...  
keycloak.init({ onLoad: "login-required", flow: 'implicit' })  
...
```

23. 🛠️ Se déconnecter et établir un **diagramme de séquence du processus d'authentification Implicit flow** sur l'application de bout en bout, en **mettant en avant les 2 requêtes étudiées précédemment**
24. 🛠️ **Comparer les flux d'authentification Standard flow et Implicit flow**, notamment leurs avantages et faiblesse. Quel flux privilégier pour une application en production ?
25. Modifier à nouveau la **configuration du client webapp-frontend** ainsi que le **code JavaScript** pour revenir au flux d'authentification **Standard flow**

→ **Faire valider les diagrammes de séquence**

Analyse des tokens JWTs

26. 🛠️ Une fois authentifié, **décoder l'access token** avec www.jwt.io et **expliquer l'utilité des claims principaux du payload qu'il contient** (`exp`, `iat`, `sub`, ...)
27. 🛠️ Calculer la **durée de validité de l'access token** en minutes



Cette durée de validité est configurable dans les **Realm settings** puis dans l'onglet **Tokens**

28. 🛠️ **Décoder l'ID token** avec www.jwt.io et **comparer les claims du payload** par rapport à ceux de l'access token
29. 🛠️ **Expliquer l'utilité de l'ID token** par rapport à l'access token

30. 🛠️ **Décoder le refresh token** avec www.jwt.io et **comparer les claims du payload** par rapport à ceux de l'access token
31. 🛠️ **Expliquer l'utilité du refresh token** par rapport à l'access token
32. 🛠️ Calculer la **durée de validité du refresh token** en minutes, et la **comparer avec celle de l'access token**

Configuration de la double-authentification (2FA)

33. Dans Keycloak, modifier l'utilisateur créé sur le realm **webapp** et ajouter dans les actions requises lors de la prochaine connexion (**Required user actions**) le fait de configurer la double-authentification avec un OTP (**Configure OTP**).
34. Se déconnecter, tenter à nouveau de se connecter sur l'application et **constater l'écran de configuration OTP**.
35. 🛠️ Copier l'image du QR code et **décoder le QR code, quel est son contenu ?**
36. En téléchargeant une application dédiée aux OTP sur votre mobile (par exemple 2FAS, Google Authenticator, ...), **configurer le code OTP en scannant le QR code avec l'application**.
37. Se déconnecter, tenter à nouveau de se connecter sur l'application et constater qu'un code OTP vous est bien demandé en plus de votre mot de passe.

JSON Web Key Sets (JWKs)

💡 Dans la fonction **api_account** dans le **code Python de l'API** (webapp/app.py), l'utilisation de la fonction `keycloak_openid.userinfo` effectue un appel API à notre serveur Keycloak.

Sur une application en production, avec plusieurs milliers de requêtes par seconde, ceci peut **fortement ralentir les performances** de devoir faire un appel à Keycloak lors de chaque vérification de JWT.

Il est très utile de pouvoir **vérifier la validité d'un JWT sans nécessairement effectuer un appel à l'API de Keycloak**. Pour cela, nous allons utiliser les clés publiques fournies par Keycloak.

38. 🛠️ Décoder à nouveau l'access token avec www.jwt.io et **expliquer la signification du claim "kid" du header** (voir cours)
39. 🛠️ En consultant la liste des clés utilisées pour signer les JWT sur le realm **webapp** (disponible à l'adresse : <http://localhost:8080/realms/webapp/protocol/openid-connect/certs>), quel est le **type d'algorithme de chiffrement (symétrique ou asymétrique)** de la clé correspondante au **kid** du JWT ?



Cette liste de clés est exposée par Keycloak s'appelle le **JSON Web Key Set ("JWKS")**. Il s'agit de l'ensemble des clés publiques associées aux clés de chiffrement utilisées pour signer les JWT du realm. La liste est exposée publiquement (sans authentification) car elle ne contient pas d'informations sensibles (il ne s'agit que des clés publiques et non des clés privées).

Cette liste de clés publiques peut être utilisée par n'importe quelle application pour **vérifier si la signature d'un JWT est bien valide et émise par Keycloak**.

Attention : en plus de la vérification de la signature, il faut toujours contrôler les autres claims du JWT, comme la date d'expiration (claim **exp**) pour s'assurer qu'il est toujours valide.

40. 🛠️ Quels sont les **avantages et faiblesses de cette méthode de vérification**, plutôt que de demander à l'API Keycloak si le JWT fourni est valide ?
41. 🛠️ Modifier la fonction **api_account** dans le **code Python de l'API** (webapp/app.py) pour utiliser uniquement le package **PyJWT** afin de **vérifier la signature du JWT** fourni à partir du JWKS (<http://localhost:8080/realms/webapp/protocol/openid-connect/certs>)

→ **Faire valider le fonctionnement de cette modification**

[BONUS pour aller plus loin] Attributs personnalisés



Avec Keycloak, il est possible de définir des attributs personnalisés sur chaque utilisateur qui peuvent être ajoutés au JWT.

42. Dans Keycloak, aller dans les **Realm settings** du realm **webapp** et activer les attributs personnalisés : **Unmanaged Attributes → Only administrators can write**



Il est important de ne pas simplement sélectionner **Enabled**, sinon tous les utilisateurs pourront **modifier eux-mêmes leurs attributs personnalisés**. Si vous stockez des informations sensibles dans ces attributs, la sécurité de votre application peut être compromise.

43. Consulter les détails de l'utilisateur créé précédemment et se rendre dans l'onglet **Attributes**
44. Ajouter l'attribut (clé/valeur) de votre choix à l'utilisateur (ex : langue=fr, ecole=CPE, ...)
45. Aller sur le client **webapp-backend** créé précédemment, se rendre dans l'onglet **Client scopes** puis cliquer sur **webapp-backend-dedicated**



Cette section permet de gérer les **mappers** du client webapp-backend. Il s'agit des informations personnalisées qui seront ajoutées à chaque JWT généré par Keycloak.

46. Cliquer sur **Configure a new mapper** et sélectionner **User attribute**



Cette section permet d'indiquer à Keycloak quels attributs utilisateurs ajouter aux JWT, et de quelle manière les ajouter.

47. Compléter le nouveau mapper de la manière suivante :
- a. **Name** → nom de votre attribut personnalisé (langue, ecole, ...)
 - b. **User attribute** → nom de votre attribut utilisateur personnalisé (langue, ecole, ...)
 - c. **Token Claim Name** → nom du claim à ajouter aux JWT (langue, ecole, ...)
48. Se déconnecter et tenter à nouveau de se connecter sur l'application
49. 🛠️ Décoder à nouveau l'access token avec www.jwt.io et **montrer la présence du nouveau claim** correspondant à l'attribut personnalisé

🎉 **Le TP est terminé !**